Reference count one at the write

The counted heap, the lowering, and the typing that puts the count there

Gavin Mendel-Gleason
Scidonia

2026

Abstract

A reference-counted runtime updates an object in place exactly when its count is one. Implementations decide this at run time, because one compiled body serves owning and borrowing call sites; but the count is determined by which references the term and the surrounding structure hold, so it is a static property of the program. This paper formalises a pure core, a reference-counted target over a counted heap, and the lowering between them, and proves the pair that characterises the target's write: an owned store is destructive and keeps its location, a shared store copies. That pair reads the count of a *reference* in the term's own account of itself; a compiler reads a count of a *name*. The second half of the paper closes the gap with a typing: the write's rule asks the subject's name to be owned exactly once in the store's environment and not borrowed, and the rules that bind a name ask a written name to be used exactly once in the scope that binds it and to be introduced for, or matched out of, a construction rather than another name. The programs a second holder of the cell would need — the subject also used as the written value, an arm binding it twice, a use of it in a sibling position, and a binding of it to another name — have no typing, each beside the same-shape program that has one. A correspondence between the typing's environment and the target's account reads the typing's count of a name into the write theorem's count of a reference, the injectivity that reading needs is derived rather than assumed where a run reaches it, and the two routes meet at one step of an accepted run. Everything is mechanised in Lean 4 against the core library alone, with no evaluation by native decision and no foreign axiom. The lowering is proved total and deterministic, not correct; that is stated, and so is each thing measured or trusted rather than proved.

1 Introduction: the count at a write

Lean's values are immutable. The term `a.set! i v` denotes a new array: same length, same elements except at `i`. No term of the language can change a value that already exists, and no program can tell whether some other array happens to share a buffer with this one. The source language is pure, and the compiler is free to make it fast.

The compiler is fast in a specific way. When the array `set!` is applied to has exactly one reference, the emitted code stores into that array's buffer and returns a pointer to the same buffer. When the array has two or more references, the emitted code allocates a copy and stores into that. Both implement the same value, so nothing about correctness depends on which one is chosen. What is at stake is allocation, and therefore speed.

The count, and where it comes from. A reference-counted runtime keeps one number per cell: the number of references to it. At a write to a cell, that number is the sum of two things — the references the cell's own slots hold, plus the references held outside the heap by whatever is running. Those outside references are, at the moment of the write, the ones the term itself holds together with any handle the surrounding structure holds. That is a fact about the program

and its data, not a reading taken from the machine: an implementation performs the same sum, per site, at run time.

What a compiler reads per site. One compiled body serves an owning call site and a borrowing one, so the decision cannot be made once and for all when the body is compiled. What the compiler emits is a test, and it decides the test at each site from what it can see there: how many times a variable is used, whether two names may denote one cell, and how an update is projected through a record's fields. The test is exact when the analysis is exact, and conservative otherwise. Nothing in this is specific to arrays: the same test guards any cell of a counted heap.

The value. An in-place update avoids an allocation and reuses the buffer. It is not an optimisation of the meaning of the program — the meaning is the same either way — it is an optimisation of the run.

The crux. The condition a write would like to hold — that the cell it writes is the program's own, named nowhere else — is a fact about the *compiled code* and not about the meaning of the program. A source term has no way to state it: it is not a value, not a type, and not an equation. Getting it to hold where a programmer wants a write to land in place is therefore a *discipline* on the source, and a discipline is what this paper builds after the target is characterised: a term is typed only when its writes are expressible on the target at all, and the typing is stated as a rule of the source so that the programmer can see which terms qualify. The count the target reads is still a count in a heap; the typing says which terms put the right count there.

What this paper does, and how each claim is labelled. Sections 2 to 2.7 fix the mechanics: the counted heap, the pure source calculus, the lowering strategy, the reference-counted target, the account, and the pair of theorems that characterises the target's write. Section 3 is the typing, with its four refusals beside their typed companions. Section 4 reads the typing's count of a name into the write theorem's count of a reference, and shows why the injectivity that reading needs is necessary, and where it is derived rather than assumed. Section 5 reads the same statement at a configuration an accepted run reaches. Sections 6 and 7 state what is trusted and what is not proved, and place the work.

Each claim below carries its status. **Proved** means a theorem of the library, quoted here as a checked line range rather than transcribed; **structural** means a fact the checked sources enforce without a single theorem stating it; **measured** means an observation taken by running compiled code; **trusted** means a link this development does not prove.

2 The condition, and the pair

This section fixes everything the pair of theorems in section 2.1 names. It is the mechanics the pair rests on: a counted heap with the two branches of its write (section 2.2), the source calculus the lowering starts from (section 2.3), the lowering strategy (section 2.4), the target's runtime over the same heap (section 2.5), the account that relates a term to the references it holds (section 2.6), and the two hypotheses the account cannot supply (section 2.7).

2.1 The pair

The write is destructive when its operand is owned once and nothing else names the cell; otherwise it copies. That is the pair proved here, and it is the only claim made about the target's write.

An owned store keeps its location *Scidonia/RC/Ownership.lean, lines 559–564*

```
theorem store_owned_is_destructive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : (expand m).count r = 1) (hslot : (CHeap.slots c).count r = 0)
    (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks := by
```

A shared store copies *Scidonia/RC/Ownership.lean, lines 600–606*

```
theorem store_shared_copies {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var}
    {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : 2 ≤ (expand m).count r ∨ 1 ≤ (CHeap.slots c).count r) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup (fresh c ks) = some ks ∧
      (update c (hs ++ expand m) r ks).1.lookup r = CHeap.lookup c r ∧
      (update c (hs ++ expand m) r ks).1.count r = CHeap.count c r - 1 := by
```

`Couple` is the coupling invariant (section 2.6), `Owns` is the term-indexed account of the term's own references, `expand m` is the reference list the account stands for, and `update` is the heap's two-branch write (section 2.2). The first theorem's conclusion is the destructive branch read back: the location still holds the written slots. The second's is the copying branch: the written slots are at a location the heap did not list, the old cell keeps its slots, and the old cell loses a reference. The two hypotheses the account cannot supply — no slot and no run handle naming the cell — are named rather than hidden (section 2.7).

Read together the two theorems say: the write is in place exactly where the account counts the operand once. The second is what keeps the first from being vacuous, and section 4.2 exhibits the configuration that makes that precise: a subject a second reference names *must* copy, or the update would be visible through the alias. A statement that every store is destructive is simply false, and the pair is the honest form of the claim.

2.2 The counted heap

A cell holds the references its slots name and the count the runtime keeps for it; a heap lists the live cells.

A cell, and a heap *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 156–172*

```
structure Cell where
  slots : List Ref
  count : Nat
deriving DecidableEq, Repr

abbrev CHeap := List (Ref × Cell)
```

One reference names one cell, so the count a heap keeps at a reference is not a choice among cells.

Well-formedness, and liveness *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 329–337*

```
def CHeap.WF (c : CHeap) : Prop := (CHeap.refs c).Nodup

def CHeap.Live (c : CHeap) : Prop := c.find? (fun e => e.2.count == 0) = none
```

No live cell has count zero, because a release deallocates at zero. A cell's count is what it must be: its occurrences among the live cells' slots, plus the handles held outside the heap — the references the running term holds.

What the counts mean *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 461–462*

```
def Counted (c : CHeap) (handles : List Ref) : Prop :=
  ∀ r, CHeap.count c r = refCount (CHeap.presentation c) handles r
```

The write branches on that count: at one the cell keeps its location (and the references its old slots held are dropped, cascading while any listed cell reaches zero), otherwise a fresh cell takes the written slots and the old cell loses a reference.

The heap's write, in two branches *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 1858–1863*

```
def update (c : CHeap) (handles : List Ref) (r : Ref) (ks : List Ref) : CHeap × List Ref :=
  if CHeap.count c r = 1 then
    (CHeap.cascade (CHeap.writeSlots c r ks), handles)
  else
    ((fresh c ks, cellOf ks) :: CHeap.addOccurrences (CHeap.adjust c r (· - 1)) ks,
      fresh c ks :: dropHandle handles r)
```

The two branches, as already proved of the heap: a destructive write is read back at the same location, a copying write at a location the heap did not list.

The destructive branch *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 1945–1949*

```
theorem update_preserves_location_of_exclusive {c : CHeap} {handles : List Ref}
    (hW : CHeap.WF c) (hc : Counted c handles) {r : Ref} {ks : List Ref}
    (h1 : Exclusive (CHeap.presentation c) handles r) (hhandle : 1 ≤ handles.count r)
    (hks : r ∉ ks) :
    (update c handles r ks).1.lookup r = some ks := by
```

The copying branch *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 1973–1979*

```
theorem update_copies_of_shared {c : CHeap} {handles : List Ref} (hc : Counted c handles)
    {r : Ref} {ks : List Ref} (h2 : ¬ Exclusive (CHeap.presentation c) handles r)
    (hhandle : 1 ≤ handles.count r) (hks : r ∉ ks) :
    (update c handles r ks).1.lookup (fresh c ks) = some ks ∧
      (update c handles r ks).1.lookup r = CHeap.lookup c r ∧
      (update c handles r ks).1.count r = CHeap.count c r - 1 ∧
      fresh c ks ∉ CHeap.refs c := by
```

Status. Proved: both branches, at a heap that is well formed and at handles the counts are stated for.

2.3 The source calculus

One term type serves both calculi. Nodes are nullary, unary or binary; an n-ary node is a spine. The write is a subject, a slot index and a value.

One term type for both calculi *Scidonia/RC/Syntax.lean, lines 110–136*

```
inductive Expr where
  | var (x : Var)
  | lam (x : Var) (body : Expr)
  | app (f a : Expr)
  | bind (x : Var) (e₁ e₂ : Expr)
  | matchC (s : Expr) (c : Ctor) (ys : List Var) (body : Expr)
  | dup (x : Var) (e : Expr)
  | drop (x : Var) (e : Expr)
  | con0 (c : Ctor)
  | con1 (c : Ctor) (a : Expr)
  | con2 (c : Ctor) (a b : Expr)
  | store (s : Expr) (i : Nat) (v : Expr)
  deriving DecidableEq, Repr
```

The source, `Pure`, is the fragment with neither instruction.

The source fragment

Scidonia/RC/Syntax.lean, lines 140–159

```
inductive Pure : Expr → Prop where
  | var (x : Var) : Pure (.var x)
  | lam {x : Var} {body : Expr} (h : Pure body) : Pure (.lam x body)
  | app {f a : Expr} (hf : Pure f) (ha : Pure a) : Pure (.app f a)
  | bind {x : Var} {e₁ e₂ : Expr} (h₁ : Pure e₁) (h₂ : Pure e₂) : Pure (.bind x e₁ e₂)
  | matchC {s : Expr} {c : Ctor} {ys : List Var} {body : Expr} (hs : Pure s)
      (h : Pure body) : Pure (.matchC s c ys body)
  | con0 (c : Ctor) : Pure (.con0 c)
  | con1 {c : Ctor} {a : Expr} (h : Pure a) : Pure (.con1 c a)
  | con2 {c : Ctor} {a b : Expr} (h₁ : Pure a) (h₂ : Pure b) : Pure (.con2 c a b)
  | store {s : Expr} {i : Nat} {v : Expr} (hs : Pure s) (h : Pure v) : Pure (.store s i v)
```

The free-variable function and the closedness discipline are what make the substitution below sound: this is the discipline the source is used at.

Free occurrences, and closedness

Scidonia/RC/Syntax.lean, lines 299–316

```
def uses : Expr → List Var
  | .var x => [x]
  | .lam x body => (uses body).filter (fun y => y ≠ x)
  | .app f a => uses f ++ uses a
  | .bind x e₁ e₂ => uses e₁ ++ (uses e₂).filter (fun y => y ≠ x)
  | .matchC s _ ys body => uses s ++ (uses body).filter (fun y => y ∉ ys)
  | .dup x e => x :: uses e
  | .drop x e => x :: uses e
  | .con0 _ => []
  | .con1 _ a => uses a
  | .con2 _ a b => uses a ++ uses b
  | .store s _ v => uses s ++ uses v

def Closed (e : Expr) : Prop := uses e = []
```

Values are the canonical forms the reduction's rules match on.

Values

Scidonia/RC/Pure.lean, lines 151–161

```
inductive Value : Expr → Prop where
  | var (x : Var) : Value (.var x)
  | lam (x : Var) (body : Expr) : Value (.lam x body)
  | con0 (c : Ctor) : Value (.con0 c)
  | con1 {c : Ctor} {a : Expr} (h : Value a) : Value (.con1 c a)
  | con2 {c : Ctor} {a b : Expr} (h₁ : Value a) (h₂ : Value b) : Value (.con2 c a b)
```

The redexes: application of an abstraction, the two binders, a match on a value node, and the write read as a functional update.

The redexes

Scidonia/RC/Pure.lean, lines 698–710

```
inductive Root : Expr → Expr → Prop where
  | beta {x : Var} {body v : Expr} : Value v → Root (.app (.lam x body) v) (subst x v body)
  | bindVal {x : Var} {v body : Expr} : Value v → Root (.bind x v body) (subst x v body)
  | matchVal {s : Expr} {c : Ctor} {ys : List Var} {body : Expr} : Value s →
      nodeTag? s = some c → ys.length = (nodeFields s).length →
      Root (.matchC s c ys body) (substAll ys (nodeFields s) body)
  | storeVal {s : Expr} {i : Nat} {v n' : Expr} : Value s → Value v →
      slotUpdate s i v = some n' → Root (.store s i v) n'
```

The reduction is a redex, or a step in one of the contexts below. There is no rule inside a match's subject, a write's subject or a node's fields: the source is written so that those are values already.

The reduction *Scidonia/RC/Pure.lean, lines 716–725*

```
inductive Step : Expr → Expr → Prop where
  | root {e e' : Expr} : Root e e' → Step e e'
  | appFun {f f' a : Expr} : Step f f' → Step (.app f a) (.app f' a)
  | appArg {f a a' : Expr} : Value f → Step a a' → Step (.app f a) (.app f a')
  | bindFun {e₁ e₁' : Expr} {x : Var} {e₂ : Expr} : Step e₁ e₁' →
      Step (.bind x e₁ e₂) (.bind x e₁' e₂)
```

The contexts the source steps in *Scidonia/RC/Pure.lean, lines 837–845*

```
inductive Ectx where
  | hole
  | appL (E : Ectx) (a : Expr)
  | appR (f : Expr) (E : Ectx)
  | bindL (x : Var) (E : Ectx) (e₂ : Expr)
```

Substitution is naive, and therefore capture free only on closed arguments.

Substitution on closed arguments *Scidonia/RC/Pure.lean, lines 538–539*

```
theorem uses_subst_of_closed : ∀ (e : Expr) (x : Var) (v : Expr), Pure e → Closed v →
    uses (subst x v e) = (uses e).filter (fun y => y ≠ x)
```

The reduction is a function, which is what lets the lowering be one too.

The contractum is unique *Scidonia/RC/Pure.lean, lines 781–782*

```
theorem step_deterministic : ∀ {e e₁ : Expr}, Step e e₁ →
    ∀ {e₂ : Expr}, Step e e₂ → e₁ = e₂ := by
```

Status. Proved: determinism, and that a step preserves closedness (`step_preserves_closed`, beside the substitution law above).

2.4 The lowering

A derivation relates a source term to its target under a borrowed and an owned multiset of occurrences. Availability is about names, not multiplicities: a name used twice may be fitting at one owned occurrence, and the strategy supplies the second use with a duplication.

Availability is about names *Scidonia/RC/Lower.lean, lines 158*

```
def Fits (Δ Γ : Occs) (e : Expr) : Prop := ∀ x ∈ uses e, x ∈ Γ ∨ x ∈ Δ
```

The relation: one rule per source construct. The splits are the functions `claimEnv`, `bindEnv` and `armEnv` rather than an existential, and each rule's conclusion wraps its translation in `wrap`, the prologue that duplicates and releases.

The lowering, one rule per construct *Scidonia/RC/Lower.lean, lines 232–272*

```
inductive Lower : Occs → Occs → Expr → Expr → Prop where
  | var {Δ Γ x} (h : x ∈ Γ ∨ x ∈ Δ) : Lower Δ Γ (.var x) (wrap Γ [x] (.var x))
  | lam {Δ Γ x body body'} (h : Lower (forget Δ x) (bindEnv Γ x (uses body)) body body') :
      Lower Δ Γ (.lam x body) (wrap Γ (uses (.lam x body)) (.lam x body'))
  | app {Δ Γ f a f' a'} (hf : Lower Δ (claimEnv Γ (uses f)) f f')
      (ha : Lower Δ (claimEnv Γ (uses a)) a a') :
      Lower Δ Γ (.app f a) (wrap Γ (uses (.app f a)) (.app f' a'))
  | bind {Δ Γ x e₁ e₂ e₁' e₂'} (h₁ : Lower Δ (claimEnv Γ (uses e₁)) e₁ e₁')
      (h₂ : Lower (forget Δ x) (bindEnv Γ x (uses e₂)) e₂ e₂') :
      Lower Δ Γ (.bind x e₁ e₂) (wrap Γ (uses (.bind x e₁ e₂)) (.bind x e₁' e₂'))
  | matchC {Δ Γ s c ys body s' body'} (hs : Lower Δ (claimEnv Γ (uses s)) s s')
      (h : Lower (forgetAll Δ ys) (armEnv Γ ys (uses body)) body body') :
      Lower Δ Γ (.matchC s c ys body)
        (wrap Γ (uses (.matchC s c ys body)) (.matchC s' c ys body'))
  | con0 {Δ Γ c} : Lower Δ Γ (.con0 c) (wrap Γ [] (.con0 c))
  | con1 {Δ Γ c a a'} (h : Lower Δ (claimEnv Γ (uses a)) a a') :
      Lower Δ Γ (.con1 c a) (wrap Γ (uses (.con1 c a)) (.con1 c a'))
  | con2 {Δ Γ c a b a' b'} (h₁ : Lower Δ (claimEnv Γ (uses a)) a a')
      (h₂ : Lower Δ (claimEnv Γ (uses b)) b b') :
      Lower Δ Γ (.con2 c a b) (wrap Γ (uses (.con2 c a b)) (.con2 c a' b'))
  | store {Δ Γ s i v s' v'} (hs : Lower Δ (claimEnv Γ (uses s)) s s')
      (h : Lower Δ (claimEnv Γ (uses v)) v v') :
      Lower Δ Γ (.store s i v) (wrap Γ (uses (.store s i v)) (.store s' i v'))
```

The relation is exactly the graph of the computable strategy `lowerB` — `lowerB_sound` and `lowerB_complete` are its two directions — so the strategy's own properties are the relation's. It is total on the fragment and deterministic, and the second is why the splits are canonical: with an arbitrary split of a binding's environment the one required duplication could be placed in either subterm and the targets would differ.

Totality *Scidonia/RC/Lower.lean, lines 514–515*

```
theorem lower_total {Δ Γ : Occs} {e : Expr} (hp : Pure e) (hfit : Fits Δ Γ e) :
    ∃ e', Lower Δ Γ e e' := by
```

Determinism *Scidonia/RC/Lower.lean, lines 524–526*

```
theorem lower_deterministic {Δ Γ : Occs} {e e₁ e₂ : Expr} (h₁ : Lower Δ Γ e e₁)
    (h₂ : Lower Δ Γ e e₂) : e₁ = e₂ := by
  have hb₁ := lowerB_complete h₁
```

Status. Proved. These are properties of the translation. Nothing here says the target computes what the source computes; that is section 6.

2.5 The target and its runtime

A configuration carries the heap, the handles the run holds, an environment from variables to references, the term's own account of its references, and the term. The environment is a list whose first pair for a name is the one the name names.

The runtime environment *Scidonia/RC/Target.lean, lines 102*

```
abbrev TEnv := List (Var × Ref)
```

The term's own account is one pair per occurrence the term itself holds, and `expand` projects the references it stands for — the tags are not read by the counts.

The term's own account *Scidonia/RC/Target.lean, lines 178–182*

```
abbrev Multi := List (Var × Ref)

def expand (m : Multi) : List Ref := m.map (fun p => p.2)
```

Each instruction moves the account in step with the heap: a duplication increments the count and adds an occurrence, a release decrements, or deallocates at a count of one.

The count operations, as steps *Scidonia/RC/Target.lean, lines 541–558*

```
inductive TStep :
    CHeap → List Ref → TEnv → Multi → Expr → CHeap → List Ref → TEnv → Multi → Expr
    ↪  → Prop where
  | dup {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref} {e : Expr}
      (hρ : ρ.lookup y = some r) (hlive : r ∈ CHeap.refs c) :
      TStep c hs ρ m (.dup y e) (dup c hs r).1 hs ρ (Multi.insert m y r) e
  | dropShared {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref}
      {e : Expr} (hρ : ρ.lookup y = some r) (h2 : 2 ≤ CHeap.count c r)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.drop y e) (decRef c hs r).1 hs ρ (Multi.dropOcc m y r) e
  | dropFinal {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {r : Ref}
      {e : Expr} (hρ : ρ.lookup y = some r) (h1 : CHeap.count c r = 1)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.drop y e) (release c hs r).1 hs ρ (Multi.dropOcc m y r) e
```

The write is the heap's own `update`, given the account as outside handles in addition to the run's. Its two rules are the two branches of `update`, and the only difference between them is the count.

The two store rules *Scidonia/RC/Target.lean, lines 603–625*

```
  | storeExclusive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {i : Nat}
      {v : Expr} {c₁ : CHeap} {hs₁ : List Ref} {wv r : Ref} (hρ : ρ.lookup y = some r)
      (hv : ValRef c hs ρ v c₁ hs₁ wv) (hi : i < (CHeap.slotsAt c₁ r).length)
      (hchk : writeCheck c₁ r (storeSlots c₁ r i wv) = true) (h1 : CHeap.count c₁ r = 1)
      (hsub : (y, r) ∈ m) :
      TStep c hs ρ m (.store (.var y) i v)
        (update c₁ (hs₁ ++ expand m) r (storeSlots c₁ r i wv)).1 hs₁ ρ m (.var y)
  | storeShared {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var} {i : Nat}
      {v : Expr} {c₁ : CHeap} {hs₁ : List Ref} {wv r : Ref} (hρ : ρ.lookup y = some r)
      (hv : ValRef c hs ρ v c₁ hs₁ wv) (hi : i < (CHeap.slotsAt c₁ r).length)
      (hchk : writeCheck c₁ r (storeSlots c₁ r i wv) = true) (hsub : (y, r) ∈ m)
      (hgone : nameGone m y r = true) (h2 : CHeap.count c₁ r ≠ 1) :
      TStep c hs ρ m (.store (.var y) i v)
        (update c₁ (hs₁ ++ expand m) r (storeSlots c₁ r i wv)).1 hs₁
        (TEnv.set ρ y (fresh c₁ (storeSlots c₁ r i wv)))
        (Multi.insert (Multi.dropOcc m y r) y (fresh c₁ (storeSlots c₁ r i wv))) (.var y)
```

Each of the two reads the subject's occurrence out of the term's own account (`hsub`), so an account that does not hold it cannot step. The position of the write in the tree arrives through the context, not through the typing: the positions the runtime steps in are the contexts below.

The positions the target steps in *Scidonia/RC/Target.lean, lines 671–685*

```
inductive TCtx where
  | hole
  | appL (E : TCtx) (a : Expr)
  | appR (f : Expr) (E : TCtx)
  | bindL (x : Var) (E : TCtx) (e₂ : Expr)
  | con1L (ct : Ctor) (E : TCtx)
  | con2L (ct : Ctor) (E : TCtx) (b : Expr)
  | con2R (ct : Ctor) (a : Expr) (E : TCtx)
```

A step inside a well-formed context is a step *Scidonia/RC/Target.lean, lines 719–722*

```
theorem tstep_of_plug : ∀ {E : TCtx} {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {r e' : Expr} {c' : CHeap} {hs' : List Ref} {ρ' : TEnv} {m' : Multi},
    TCtx.WF E → TStep c hs ρ m r c' hs' ρ' m' e' →
      TStep c hs ρ m (E.plug r) c' hs' ρ' m' (E.plug e') := by
```

Status. Proved: that the congruence rules are these contexts one by one, which `tstep_of_plug` states for a well-formed context.

2.6 The account

Exact tagging: each pair names the reference its variable is bound to, not merely some reference. The weaker reading admits a mis-tagged pair and makes the theorems below arithmetic.

Exact tagging *Scidonia/RC/Ownership.lean, lines 116*

```
def Bound (ρ : TEnv) (m : Multi) : Prop := ∀ p ∈ m, ρ.lookup p.1 = some p.2
```

The account is of the term: each consuming instruction reads its occurrence out of it, and a store reads the occurrence its subject names.

The account is of the term *Scidonia/RC/Target.lean, lines 417–435*

```
inductive Owns (ρ : TEnv) : Multi → Expr → Prop where
  | drop {m : Multi} {y : Var} {r : Ref} {e : Expr} (hρ : ρ.lookup y = some r)
      (hmem : (y, r) ∈ m) : Owns ρ m (.drop y e)
  | bindVar {m : Multi} {x z : Var} {r : Ref} {e₂ : Expr} (hρ : ρ.lookup z = some r)
      (hmem : (z, r) ∈ m) (hheld : ∀ p ∈ m, p.1 = x → p.2 = r) :
      Owns ρ m (.bind x (.var z) e₂)
  | appVar {m : Multi} {x z : Var} {r : Ref} {body : Expr} (hρ : ρ.lookup z = some r)
      (hmem : (z, r) ∈ m) (hheld : ∀ p ∈ m, p.1 = x → p.2 = r) :
      Owns ρ m (.app (.lam x body) (.var z))
  | store {m : Multi} {y : Var} {r : Ref} {i : Nat} {v : Expr} (hρ : ρ.lookup y = some r)
      (hmem : (y, r) ∈ m) : Owns ρ m (.store (.var y) i v)
```

The coupling invariant: the term's own references are handles. That is the whole content of the middle conjunct.

The coupling invariant *Scidonia/RC/Ownership.lean, lines 161–162*

```
def Couple (c : CHeap) (hs : List Ref) (ρ : TEnv) (m : Multi) : Prop :=
  CHeap.WF c ∧ Counted c (hs ++ expand m) ∧ Bound ρ m
```

Every step preserves it, contexts included.

The invariant is preserved *Scidonia/RC/Ownership.lean, lines 489–491*

```
theorem couple_step {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {e : Expr}
    {c' : CHeap} {hs' : List Ref} {ρ' : TEnv} {m' : Multi} {e' : Expr}
    (h : TStep c hs ρ m e c' hs' ρ' m' e') (hc : Couple c hs ρ m) : Couple c' hs' ρ' m' := by
```

Status. Proved, for the relation's steps.

2.7 The two hypotheses, and where they come from

`store_owned_is_destructive` cannot derive, from the account, that no heap slot and no run handle names the cell. Both are what a nested update's take-out provides: at a write the run resolves a cell, takes the element out of the slot the write lands at, holds it as a handle, and checks that cell before recursing — so the addressed cell has no slot naming it, and the run's own handle is the only name on it.

The focus check is what the pair names *Scidonia/FracPerm/StateVar/Run.lean, lines 1019–1021*

```
theorem exclusive_head {c : CHeap} {hs : List Ref} {r : Ref}
    (hes : Exclusive (CHeap.presentation c) hs r) (hhead : hs.head? = some r) :
    (CHeap.slots c).count r = 0 ∧ hs.count r = 1 ∧ hs.tail.count r = 0 := by
```

The take-out also pairs the extracted occurrence with a handle and drops that handle before the focus check, which is what leaves the cell exclusive again at the count of one the write needs.

The take-out's own arithmetic *Scidonia/FracPerm/StateVar/RCHeap.lean, lines 2082–2085*

```
theorem takeout_dropped_count_one {c : CHeap} {e : Ref} (hc : Counted c [e, e])
    (h0 : (CHeap.presentation c).occurrences e = 0) :
    (decRef c [e, e] e).1.count e = 1 ∧
      Exclusive (CHeap.presentation (decRef c [e, e] e).1) (decRef c [e, e] e).2 e := by
```

The mechanisation names the two surviving facts among the theorem's hypotheses rather than deriving them, and cites the take-out lemmas that supply them. section 5 reads the same two facts off a write step of an accepted run.

3 The typing

The pair of section 2.1 counts a *reference* in the term's own account. A compiler counting at a site counts the occurrences of a *name* in the environment that site is compiled in. The two agree only because the target's accounts happen to be injective on references, and nothing in the source language says they are. What the source needs, and what this section builds, is a judgement that admits a write only where the environment gives its subject exactly one occurrence, and that refuses the shapes where a second holder of the cell exists.

The judgement is the lowering's own environment discipline read as a typing: the same clauses over the same canonical splits, one rule per source construct, no target term, the write's clause strengthened to what a destructive write needs, and two further conditions on the name a write's subject carries.

3.1 The write's clause

The write's rule is the only place where multiplicity and not membership is read. It requires the subject's variable to be owned exactly once in the environment of the store and not borrowed.

A variable is typed where an environment supplies it *Scidonia/RC/Typing.lean, lines 255–264*

```
inductive Typed : Occs → Occs → Expr → Prop where
  | var {Δ Γ : Occs} {x : Var} (h : x ∈ Γ ∨ x ∈ Δ) : Typed Δ Γ (.var x)
  | lam {Δ Γ : Occs} {x : Var} {body : Expr}
      (hlin : writes x body = true → (uses body).count x = 1)
      (h : Typed (forget Δ x) (bindEnv Γ x (uses body)) body) : Typed Δ Γ (.lam x body)
```

The write's clause *Scidonia/RC/Typing.lean, lines 299–301*

```
  | store {Δ Γ : Occs} {y : Var} {i : Nat} {v : Expr} (hown : Γ.count y = 1)
      (hborrow : y ∉ Δ) (hv : Typed Δ (claimEnv Γ (uses v)) v) :
      Typed Δ Γ (.store (.var y) i v)
```

`ÎŞ.count y = 1` is a count of the subject's *name* in the store's environment, and `y âĹĽ ÎŤ` says the name is not borrowed there. Three readings of one judgement follow from it, and the first is the one the theorems above are read through: every write in a typed term has, at the environment the typing assigns its position, a subject owned exactly once there and not borrowed. That is the traversal `WriteOwned`, whose recursion is the typing's own.

Every write's subject is owned once, carried by a traversal *Scidonia/RC/Typing.lean,*

lines 313–331

```
def WriteOwned (Δ Γ : Occs) : Expr → Prop
  | .var _ => True
  | .lam x body => WriteOwned (forget Δ x) (bindEnv Γ x (uses body)) body
  | .app f a =>
      WriteOwned Δ (claimEnv Γ (uses f)) f ∧ WriteOwned Δ (claimEnv Γ (uses a)) a
  | .bind x e₁ e₂ =>
      WriteOwned Δ (claimEnv Γ (uses e₁)) e₁
        ∧ WriteOwned (forget Δ x) (bindEnv Γ x (uses e₂)) e₂
  | .matchC s _ ys body =>
      WriteOwned Δ (claimEnv Γ (uses s)) s
        ∧ WriteOwned (forgetAll Δ ys) (armEnv Γ ys (uses body)) body
  | .dup _ _ => True
  | .drop _ _ => True
  | .con0 _ => True
  | .con1 _ a => WriteOwned Δ (claimEnv Γ (uses a)) a
  | .con2 _ a b =>
      WriteOwned Δ (claimEnv Γ (uses a)) a ∧ WriteOwned Δ (claimEnv Γ (uses b)) b
  | .store (.var y) _ v => Γ.count y = 1 ∧ y ∉ Δ ∧ WriteOwned Δ (claimEnv Γ (uses v)) v
  | .store _ _ _ => False
```

Status. Proved. `typed_write_owned` carries the traversal over a typed term, and `typed_lowers` shows a typed term is one the lowering strategy answers from the same environments: the typing is a restriction of the discipline the lowering walks, and not a second discipline beside it.

3.2 The two conditions a written name carries

The clause above reads the environment *at the store*, and an environment is a filter of its parent's, so a name's occurrences are claimed separately by each position that uses them: shares overlap, and a sibling's use of the subject's name is not counted at the store. A term whose write's subject is also used in a sibling position therefore meets the write's clause — the store's own occurrence is the subject's — while the program as a whole uses the subject's name twice, and the lowering supplies the sibling's use with a duplication. Read at the store's environment alone, the typing cannot see the program's multiplicity, so two further conditions are needed, and both are side conditions of the rules that *bind* the name.

The first: a written name is used exactly once in the scope that binds it, and it is introduced for, or matched out of, a construction rather than another name. For a binding, both conditions sit on the rule.

Both conditions, on a binding *Scidonia/RC/Typing.lean, lines 273–277*

```
  | bind {Δ Γ : Occs} {x : Var} {e₁ e₂ : Expr}
      (hlin : writes x e₂ = true → (uses e₂).count x = 1)
      (hbuild : writes x e₂ = true → BuiltB e₁ = true)
      (h₁ : Typed Δ (claimEnv Γ (uses e₁)) e₁)
      (h₂ : Typed (forget Δ x) (bindEnv Γ x (uses e₂)) e₂) : Typed Δ Γ (.bind x e₁ e₂)
```

For a matching arm, they are carried for each of the arm's binders, and the construction the name is matched out of is the matched subject.

Both conditions, on an arm *Scidonia/RC/Typing.lean, lines 282–287*

```
  | matchC {Δ Γ : Occs} {s : Expr} {c : Ctor} {ys : List Var} {body : Expr}
      (hlin : ∀ y ∈ ys, writes y body = true → (uses body).count y = 1)
      (hbuild : ∀ y ∈ ys, writes y body = true → BuiltB s = true)
      (hs : Typed Δ (claimEnv Γ (uses s)) s)
      (h : Typed (forgetAll Δ ys) (armEnv Γ ys (uses body)) body) :
      Typed Δ Γ (.matchC s c ys body)
```

The first condition is what makes the *program* use the bound name once in the scope that binds it, rather than counting only the store's share of the environment; `writes` is the traversal

the condition reads, true at a write whose subject is the free variable `x` and stopped by a binder of `x`. The second says the written name comes from a construction rather than another name: a binding's written name is bound to a node the program built and never to a variable, which would make the written name a second name beside that variable whatever cell it names. `typed_write_linear` and `typed_write_built` carry the two conditions over a typed term, as traversals in the style of `WriteOwned`.

Status. Proved: the conditions are side conditions of the binding rules and are carried over typed terms, so the write's own clause and the two name conditions are three readings of one judgement.

3.3 The refusals, and their typed companions

Each shape in which a second holder of the cell exists has no typing, and each sits beside the same-shape program that has one. The eight programs are these.

The eight programs *Scidonia/RC/Typing.lean, lines 541–572*

```
def dupValue : Expr := .lam 0 (.con1 7 (.store (.var 0) 0 (.var 0)))

def ownedValue : Expr := .lam 0 (.con1 7 (.store (.var 0) 0 (.con0 9)))

def dupArm : Expr := .matchC (.con1 5 (.con0 5)) 0 [0, 0] (.store (.var 0) 0 (.con0 9))

def ownedArm : Expr := .matchC (.con1 5 (.con0 5)) 0 [0] (.store (.var 0) 0 (.con0 9))

def siblingUse : Expr := .lam 0 (.con1 7 (.con2 8 (.store (.var 0) 0 (.con0 9)) (.var 0)))

def siblingOwned : Expr := .lam 0 (.con1 7 (.con2 8 (.store (.var 0) 0 (.con0 9)) (.con0 10)))

def aliasBind : Expr := .bind 0 (.var 1) (.store (.var 0) 0 (.con0 9))

def builtBind : Expr := .bind 0 (.con1 5 (.con0 5)) (.store (.var 0) 0 (.con0 9))
```

The subject is also the written value. The store's environment carries two occurrences of the subject's name, and the write's clause asks for one.

The subject used twice: no typing *Scidonia/RC/Typing.lean, lines 577*

```
theorem dupValue_no_typing : ¬ Typed [] [] dupValue := by
```

The same shape, the subject used once *Scidonia/RC/Typing.lean, lines 597*

```
theorem ownedValue_typed : Typed [] [] ownedValue := by
```

An arm binds the subject twice. The subject is supplied twice by the binder list, so the body uses the bound name twice.

The arm binding it twice: no typing *Scidonia/RC/Typing.lean, lines 588*

```
theorem dupArm_no_typing : ¬ Typed [] [] dupArm := by
```

The same arm, binding it once *Scidonia/RC/Typing.lean, lines 605*

```
theorem ownedArm_typed : Typed [] [] ownedArm := by
```

A sibling use. The store's own clause is met — the sibling's use is not counted at the store — and the program still uses the subject's name twice, so the first name condition refuses it. This is the refutation that the store's own environment alone cannot give, and its companion writes through the name and does not use it elsewhere.

A sibling use: no typing *Scidonia/RC/Typing.lean, lines 626*

```
theorem sibling_use_no_typing : ¬ Typed [] [] siblingUse := by
```

The same program, using the name once *Scidonia/RC/Typing.lean, lines 642*

```
theorem sibling_use_owned : Typed [] [] siblingOwned := by
```

An alias binding. A binding's written name is bound to another name, so the written name is a second name beside it whatever cell it names. The refutation is at every environment, because the condition is a statement about the term, and its companion binds the name to a node the program built.

A binding to another name: no typing *Scidonia/RC/Typing.lean, lines 655–656*

```
theorem alias_bind_no_typing {Δ Γ : Occs} {x z : Var} {i : Nat} {v : Expr} :
    ¬ Typed Δ Γ (.bind x (.var z) (.store (.var x) i v)) := by
```

The same binding, to a construction *Scidonia/RC/Typing.lean, lines 667*

```
theorem built_bind_typed : Typed [] [] builtBind := by
```

The reading. The shapes where a second holder of the cell exists have *no* typing, and each beside its companion shows why: the companions differ from the refuted programs in nothing but the multiplicity of the subject's name or the shape of what binds it. A write is therefore only expressible where the cell is the program's own — which is exactly the condition the target's destructive branch asks the account for, in the source's own vocabulary. `dup_no_typing` and `drop_no_typing` record the other half: a resource instruction has no rule, so a typed term mentions neither.

3.4 What the conditions do not reach

The two conditions are stated of the rules that bind a name *within* the term, and a written name may be one the term binds nowhere. As the module says of its own limitation: a subject that is a free variable of the term is the environment's, and the store rule admits it wherever the environment holds it once and it is not borrowed; an abstraction's name is bound to the argument an application supplies, and no side condition of an abstraction or an application can see that argument's shape; and a matching arm's binder names a *field* of the matched cell, and a field that is itself a name is a second name for that name's cell. Those cases — together with the write reached through a nested update, where the cell's count of one and the absence of any other name for it are facts the take-out provides and `store_owned_is_destructive` names among its hypotheses rather than deriving — remain outside the judgement.

What is proved is the name side of the write: where the term itself binds the name a subject carries, that name is used exactly once in the scope that binds it and is introduced for, or matched out of, a construction.

Status. Proved, of the rules the term itself carries, and explicitly not of the three cases above.

3.5 The typing is a statement about the source

A target-side sufficient condition — a judgement over the target's own relation, asking at a store for the two hypotheses the write theorem needs — is the obvious other route, and it is not the one taken. It is refuted by the lowering's own prologue: `Runs` demands of a binding whose bound term is a value that the continuation run at every reference the binder could take, and at the well-built reference the account counts that reference three times.

A target-side condition, and what it costs *Scidonia/RC/Discipline.lean, lines 403–405*

```
theorem runs_store_owned {ρ : TEnv} {m : Multi} {y : Var} {i : Nat} {v : Expr}
    (h : Runs ρ m (.store (.var y) i v)) :
    Owns ρ m (.store (.var y) i v) ∧ ∃ r, ρ.lookup y = some r ∧ (expand m).count r = 1 := by
```

A source term the strategy walks and its canonical lowering are one pair of witnesses to that:

A source term and its canonical lowering *ScidoniaTest/Target.lean, lines 505–510*

```
def source : Expr := .bind 0 (.con2 7 (.var 1) (.var 2)) (.store (.var 0) 0 (.var 2))

def lowered : Expr :=
  .dup 2 (.drop 0 (.bind 0 (.con2 7 (.var 1) (.var 2)) (.store (.var 0) 0 (.var 2))))
```

The source is typed *ScidoniaTest/Target.lean, lines 597*

```
theorem source_typed : Typed [] [0, 1, 2] source := by
```

The lowering is not judged *ScidoniaTest/Target.lean, lines 666*

```
theorem lowered_not_runs : ¬ Runs env0 [(0, 0), (1, 1), (2, 2)] lowered := by
```

The typing holds at the environments the strategy walks, and the target-side judgement does not hold at the lowering — which is why the typing is stated at the source, where the rules that bind a written name are visible, rather than at the target, where the prologue's duplication has already hidden the program's multiplicity behind an account.

4 From names to the count

The typing counts names in an environment; the write theorem of section 2.1 counts a reference in the term's own account. The link between them is a correspondence between the target's environment `ΪĄ`, the term's own account `m` and the typing's environment `ÎŞ`.

4.1 The correspondence

The correspondence has three clauses: every pair of the account names the reference the environment binds its name to, the account's multiplicity at each name is the typing's count of that name, and the account's tags are injective on references.

The correspondence *Scidonia/RC/Correspondence.lean, lines 129–136*

```
structure Corr (ρ : TEnv) (m : Multi) (Γ : Occs) : Prop where
  bound : Bound ρ m
  name : ∀ x, countName m x = Γ.count x
  inj : RefInj m
```

The first two clauses are already enough for half of the write theorem's premises: the account's occurrence of the subject's name and the environment's reading of it give the term-indexed account `Owns`. This half needs no injectivity, and it is why the countermodel of section 4.2 separates on the count of the reference and not on the occurrence.

Half the bridge, without injectivity *Scidonia/RC/Correspondence.lean, lines 292–295*

```
theorem owns_of_correspondence {ρ : TEnv} {m : Multi} {Γ : Occs} {y : Var} {r : Ref}
    {i : Nat} {v : Expr} (hbound : Bound ρ m) (hname : countName m y = Γ.count y)
    (hone : Γ.count y = 1) (hlook : ρ.lookup y = some r) :
    Owns ρ m (.store (.var y) i v) := by
```

The third clause is what turns the typing's count of a *name* into the write theorem's count of a *reference*: the account has one pair for the name, and an injective account puts no second name on that pair's reference.

The bridge *Scidonia/RC/Correspondence.lean, lines 305–307*

```
theorem bridge {ρ : TEnv} {m : Multi} {Γ : Occs} {y : Var} {r : Ref} {i : Nat} {v : Expr}
    (h : Corr ρ m Γ) (hlook : ρ.lookup y = some r) (hone : Γ.count y = 1) :
    Owns ρ m (.store (.var y) i v) ∧ (expand m).count r = 1 := by
```

Composed with the write theorem, a store typed at `ÎŞ` whose environment counts its subject once, at a configuration corresponding in the sense of `Corr`, is destructive.

A typed store is destructive *Scidonia/RC/Correspondence.lean, lines 435–440*

```
theorem store_destructive_of_typed {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {Δ Γ : Occs} {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref}
    (hc : Couple c hs ρ m) (hcorr : Corr ρ m Γ) (hlook : ρ.lookup y = some r)
    (h : Typed Δ Γ (.store (.var y) i v))
    (hslot : (CHeap.slots c).count r = 0) (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks := by
```

Its hypotheses `hslot`, `hrun` and `hks` are the write theorem's own, named there as what the discipline's take-out provides and what the write rule's check demands; the conclusion is that theorem's. `bridge` reads the typing's own write clause off the derivation (`Typed.store_count`), so no injectivity is an input of this theorem's either — it is supplied by `Corr`, and section 4.3 is where that clause comes from at a run.

Status. Proved.

4.2 The injectivity is necessary

The injectivity clause is the one neither the typing nor the coupling supplies, and the configuration that separates them is two names naming one reference: the store is typed there, the account is `Bound` and inside `Couple`, the account's multiplicity at the subject's name is the typing's count of it, and the reference is still counted twice. So the count of one does not follow from the judgement and the invariant together.

The whole invariant, and the count of two *Scidonia/RC/Correspondence.lean, lines 404–408*

```
theorem couple_does_not_give_count_one :
    Couple ([(0, { slots := [], count := 2 })] : CHeap) [] aliasEnv aliasM
      ∧ Typed ([] : Occs) ([0,1] : Occs) (.store (.var 0) 0 (.con0 9))
      ∧ countName aliasM 0 = ([0,1] : Occs).count 0
      ∧ (expand aliasM).count (0 : Ref) ≠ 1 :=
```

Only `RefInj` refuses that configuration, and the module exhibits the refusal beside the three facts that do not (`alias_typed`, `alias_couple`, `alias_not_inj`, with the reference count of two read at `alias_ref_count_two`). What would supply the clause on the environment's side is a distinctness or freshness condition on the environment the program starts from; the next subsection is where the run supplies it instead.

Status. Proved: the countermodel is a theorem, and so is the failure of the count of one at it.

4.3 The injectivity at a take-out

At a nested write the injectivity is not an input. The run's counting at the take-out's level, beside the coupling's own counting of one heap, forces the account's references to be the one occurrence the take-out extracted.

Two countings force the account's references *Scidonia/RC/Transfer.lean, lines 233–235*

```
theorem expand_eq_singleton_of_counted {c : CHeap} {hs : List Ref} {m : Multi} {e : Ref}
    (hrun : Counted c (e :: hs)) (hcouple : Counted c (hs ++ expand m)) :
    expand m = [e] := by
```

And an account whose references are a single occurrence is injective whatever names its pairs carry.

And one occurrence is injective *Scidonia/RC/Transfer.lean, lines 264–265*

```
theorem refInj_of_counted {c : CHeap} {hs : List Ref} {m : Multi} {e : Ref}
    (hrun : Counted c (e :: hs)) (hcouple : Counted c (hs ++ expand m)) : RefInj m :=
```

So at a take-out the injectivity is *derived*, not assumed: the take-out pins the account's size, and a one-pair account cannot name a reference twice. The boundary is sharp, and the library states it too: two accounts with the same references are one configuration to the run's side — the same count at every heap and handle list — and one of them names a reference twice.

The run's side does not determine the injectivity *Scidonia/RC/Transfer.lean, lines 338–340*

```
theorem refInj_not_determined :
    ∃ m₁ m₂ : Multi, expand m₁ = expand m₂ ∧ RefInj m₁ ∧ ¬ RefInj m₂ ∧
      (∀ (c : CHeap) (hs : List Ref), Counted c (hs ++ expand m₁) ↔ Counted c (hs ++ expand
      ↪  m₂)) := by
```

Status. Proved. The reading is exact in both directions: injectivity is necessary (section 4.2) and it is not a function of the references alone.

5 Reading a run through it

The composition above is a statement about a configuration, and a run is a sequence of them. This section reads the two routes at one step: the typing's route to the write theorem, and the run model's own write step.

At a nested write the take-out has emptied the slot the write lands at and holds the extracted occurrence as a handle, so the run's handle list is that occurrence followed by the outer handles while the coupling's split is the outer handles followed by the account's references. The two read the same references exactly when the account is the extracted occurrence, and that is the coupling the take-out hands the nested write.

The coupling the take-out hands the nested write *Scidonia/RC/Transfer.lean, lines 415–420*

```
theorem couple_of_takeout {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {r : Ref}
    {i : Nat} {e : Ref} (hW : CHeap.WF c) (hcounted : Counted c hs)
    (hhead : hs.head? = some r) (hex : Exclusive (CHeap.presentation c) hs r)
    (hlook : (CHeap.slotsAt c r)[i]? = some e)
    (hbound : Bound ρ m) (hsize : expand m = [e]) :
    Couple (held c r i e) hs ρ m := by
```

The account is read *out of* the run's handle list rather than added beside it, which is why the coupling's handles are the run's own (`couple_at_run_handles_is_empty` at the coupling's split). The account's identity is that theorem's hypothesis, since it is the coupling being built; where an already-built coupling is at hand the identity is derived instead, from that coupling's own counting beside the run's (section 4.3).

A write step of an accepted run, read through the coupling *Scidonia/RC/Transfer.lean,*

lines 507–519

```
theorem writeStep_nested_destructive_of_couple {root : Ref} {c : CHeap} {hs : List Ref}
    {vs : List Scidonia.FracPerm.StateVar.Value} {ρ : TEnv} {m : Multi} {i : Nat}
    {p : Path} {ks : List Ref}
    {e : Ref} {c' : CHeap} {hs' : List Ref} {Γ Δ : Occs} {y : Var} {j : Nat} {v : Expr}
    (hacc : Accept root c hs vs)
    (hstep : writeStep c hs root (i :: p) ks = some (c', hs'))
    (hlook : (CHeap.slotsAt c root)[i]? = some e)
    (hbound : Bound ρ m) (hsize : expand m = [e]) (hks : e ∉ ks)
    (hname : ∀ x, countName m x = Γ.count x) (hlookup : ρ.lookup y = some e)
    (hstore : Typed Δ Γ (.store (.var y) j v)) :
    Couple (held c root i e) hs ρ m ∧
      (update (held c root i e) (e :: hs) e ks).1.lookup e = some ks ∧
      hs' = hs ∧ ∃ t, resolve c root (i :: p) = some t ∧ c'.lookup t = some ks := by
```

A write step of an accepted configuration, its take-out, the account through the coupling, and the schema's own checks are the write theorem's two premises, so the destructive branch at the nested subject is a statement about that step, with `Run.writeStep_spec`'s own result beside it. The correspondence's injectivity clause is built from the two countings rather than carried (`store_destructive_of_corr`). At a write whose path's level is its last, the recursion's entry is the write's own subject and the leaf's check supplies the write rule's premise on the written slots, so the two routes' conclusions are about one cell.

The two routes at one cell *Scidonia/RC/Transfer.lean, lines 540–552*

```
theorem writeStep_subject_destructive_of_couple {root : Ref} {c : CHeap} {hs : List Ref}
    {vs : List Scidonia.FracPerm.StateVar.Value} {ρ : TEnv} {m : Multi} {i : Nat}
    {ks : List Ref} {e : Ref}
    {c' : CHeap} {hs' : List Ref} {Γ Δ : Occs} {y : Var} {j : Nat} {v : Expr}
    (hacc : Accept root c hs vs)
    (hstep : writeStep c hs root [i] ks = some (c', hs'))
    (hlook : (CHeap.slotsAt c root)[i]? = some e)
    (hbound : Bound ρ m) (hsize : expand m = [e])
    (hname : ∀ x, countName m x = Γ.count x) (hlookup : ρ.lookup y = some e)
    (hstore : Typed Δ Γ (.store (.var y) j v)) :
    Couple (held c root i e) hs ρ m ∧
      (update (held c root i e) (e :: hs) e ks).1.lookup e = some ks ∧
      c'.lookup e = some ks := by
```

What this does, and what it does not. It links the typing's route to the run's route at one step of an accepted run. The induction over a whole program's steps is the run model's own (`Run.every_write_step_keeps_its_address`, the per-run statement that a write's address holds the written slots afterwards, and `Run.writeStep_lands`, its per-step form), and neither of those is read through the coupling. What is not built is that induction *carrying the coupling*: the run model's configurations carry no term, so the account the coupling relates to is the caller's, and its term-side facts — that the account's pairs name what the environment binds them to, and that the account is the extracted occurrence — are supplied at the step rather than carried through the run.

6 What is trusted, and what is not proved

What is trusted. Three links, each with an exact boundary.

The take-out's contribution. That a nested update's cell has no slot and no handle naming it is a property of the nesting, and is taken here as the two named hypotheses of `store_owned_is_destructive`; section 2.7 is where the run supplies them, and section 5 is where they are read off a step.

The compiler's lowering. That a compiler turns a language's write paths into the shape modelled here is not proved. In the accompanying repository that link is *checked* — structurally,

and not by a theorem: the check reads the post-reference-counting intermediate representation the pinned toolchain emits and asserts the shape on one executing path, failing the build when it does not hold.

The runtime's primitive. The branch that stores in place is a runtime test whose meaning is documented in the runtime and not modelled here. The counted model states the predicate a write tests.

The predicate a write tests, at a count *Scidonia/FracPerm/StateVar/RefCount.lean, lines 720*

```
def IsExclusive (rc : Int32) : Prop := isSt rc ∧ rc = 1
```

The model's count of one is not the machine's test in either direction, and the difference is stated where it matters: the count's module says of the store's run-time primitive that its effect on the count is an *assumption* at the one place the development needs it, labelled as such, and its write-path theorem carries exactly those assumptions as hypotheses.

The write path's hypotheses are assumptions *Scidonia/FracPerm/StateVar/RefCount.lean, lines 819–823*

```
theorem takeOut_refCount_eq_one (rc0 rc1 rc2 : Int32)
    (hcell : refCount rc0 = 1)
    (hhand : refCount rc1 = refCount rc0 + 1)
    (hrelease : refCount rc2 = refCount rc1 - 1) :
    refCount rc2 = 1 := by
```

The in-place check. The check a client can turn on is instrumented and measured, not proved. In the logic it is the identity, so no specification mentions it; its compiled implementation reports and exits when the object it is given is not exclusively owned.

The in-place check: the identity in the logic *Scidonia/FracPerm/InPlace.lean, lines 92–93*

```
@[implemented_by assertExclusiveImpl]
def assertExclusive {σ : Type} (site : @& String) (s : σ) : σ := s
```

It is measured on compiled code by the repository's own harness, which runs a program under the check and reports the sites where an update would have copied. That an update copies at such a site is a fact about the run that was observed, not a theorem about the program.

The access-shape limit. The store of section 2.3 addresses one index into a cell, and depth arrives through the term: the target's write rules are read at the cell a variable names, and a nested write's path is a context over the relation rather than a form of the term. A nested write's uniqueness is therefore supplied by the take-out (section 2.7) and not by the typing, which is the honest scope of section 3: the typing speaks of the writes the term addresses directly.

What is not proved. The lowering's semantic correctness: no theorem says the target computes what the source computes, or that it is garbage free. What is proved of the lowering is totality and determinism. The declarative relation of the literature is not the one formalised here: the strategy's splits are canonical, for determinism, so the results are properties of *this* strategy, which is what a compiler implements. And nothing here is about a compiled program: the references are names in a list, not addresses.

7 Mechanisation, and related work

The library publishes 694 public theorems under `Scidonia/`, of which 17 are vendored from Lean's own model of the count. This paper quotes 65 panels of it, and every one is a line range of the source rather than a transcription: the range is generated into a frame with its comments removed, a `\%!check:` directive names a fragment the range must contain, and `check-listings.py` fails when either has drifted. The counts the prose states are read from a generated file, so a count that moves with the library is regenerated rather than corrected by hand.

The development is a few thousand lines of Lean 4 against the core library only, with no mathematics library dependency. There is no `sorry`, no evaluation by native decision, and every representative theorem's axiom list is asserted, so a proof that acquired a foreign axiom fails the build. Two design facts the formalisation forced are recorded as toolchain facts: terms are a flat inductive, because a nested one cannot be inducted on or derive decidable equality; and an evaluated check needs a structural recursion, because the kernel does not reduce a well-founded one.

Reference counting with reuse. In-place update of a shared value, and the precise counting with ownership that makes it possible, is the design of Lean 4's runtime [1] and is formalised as the linear resource calculus λ^1 in the PeTEX paper [2], which proves the counting sound and garbage free. This development follows that shape and differs where this paper's subject differs: the deliverable is the count at the write, as a pair, with the hypotheses the account cannot supply named; a typing that puts the count where a write needs it, with the shapes a second holder would need refuted; and the translation is a canonical strategy rather than the declarative relation.

Linearity, and formal models of counting. An ordinary type system does not track uniqueness: Linear Haskell [3] gives a linear arrow, and its guarantee is about the use of a value, not about a count in a heap. A formal model of reference counting for code generation appears in Férey and Shankar [4]. Neither is a count-one guarantee for a write, and neither is stated as a condition on the name a write's subject carries.

Future work. Three steps: the lowering's semantic correctness (result agreement and garbage freedom); carrying the coupling through a run's steps, which needs the run model's configurations to carry the term, so that the two routes of section 5 compose over a whole program rather than at one step; and a verified account, or a soundness theorem for the check, of the compiler's pass.

References

[1] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: reference counting optimized for purely functional programming. In *Proceedings of the 31st Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2019. arXiv:1908.05647.

[2] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. PeTEX: garbage free reference counting with reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*, 2021. Extended version: Microsoft Research Technical Report MSR-TR-2020-42, which contains the proofs.

[3] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear Haskell: practical linearity in a higher-order polymorphic language. In *Proceedings of the 45th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '18)*, 2018.

[4] Gaspard Férey and Natarajan Shankar. Code generation using a formal model of reference counting. In *NASA Formal Methods (NFM 2016)*, pages 150–165, 2016.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.

[6] Gavin Mendel-Gleason. The mechanisation of this paper, in the Scidonia `lean` repository: `Scidonia/RC/`, the counted heap in `Scidonia/FracPerm/StateVar/RCHeap.lean`, the run model in `Scidonia/FracPerm/StateVar/Run.lean`, the check in `Scidonia/FracPerm/InPlace.lean`, and the checks in `ScidoniaTest/Typing.lean`,

`ScidoniaTest/Correspondence.lean`, `ScidoniaTest/Transfer.lean` and `ScidoniaTest/Target.lean`. https://github.com/scidonia/lean-examples.